(چند دقیقه یا حتی چند ساعت) یا تقریباً همیشه اشغال باشد. در چنین مواردی، مشغله‌ی انتظار فوق‌العاده ناکارآمد است.

۶-۷ مانیتور

هرچند استفاده از سمافورها، راهکار مناسب و مؤثری برای همگامی پردازه فراهم می‌کند، استفاده‌ی نادرست آنها می‌تواند موجب خطاهای زمانی شود که تشخیص آنها دشوار است، چرا که این خطاها فقط با وقوع دنباله‌های اجرایی خاصی پیش می‌آیند، و این دنباله‌ها هم همیشه رخ نمی‌دهند.

نمونه‌ای از این خطاها را در استفاده از یک متغیر count در راه‌حل خودمان برای مسأله‌ی تولیدکننده–مصرف‌کننده دیدیم (بخش ۶-۱). در آنجا، مسأله‌ی زمانی به ندرت پیش می‌آید و حتی در آن صورت هم مقدار count مقبول به نظر می‌رسد—اختلاف فقط ۱ واحد است. با این وجود، روشن است که راه‌حل مقبولی نیست. به همین دلیل بود که قفل‌های انحصار متقابل و سمافورها برای اولین بار معرفی شدند.

متأسفانه، این قبیل خطاهای زمانی باز هم در استفاده از قفل‌های انحصار متقابل و یا سمافورها امکان‌پذیر هستند. برای روشن‌شدن مطلب، راه‌حل سمافور مسأله‌ی بخش بحرانی را بازبینی خواهیم کرد. تمامی پردازه‌ها متغیر سمافور دودویی mutex را به اشتراک می‌گیرند که در ابتدا به ۱ مقداردهی می‌شود. هر یک از پردازه‌ها بایستی پیش از ورود به بخش بحرانی wait(mutex) و پس از خروج signal(mutex) را اجرا کنند. اگر این دنباله مشاهده نشود، این امکان وجود دارد که دو پردازه همزمان به بخش بحرانی خود وارد شوند. در ادامه، مشکلات متعددی را که ممکن است دیده شوند بررسی می‌کنیم. توجه داشته باشید که این مشکلات حتی ممکن است به خاطر رفتار نادرست یک تک‌پردازه هم پیش بیاید. این شرایط می‌تواند به خاطر خطای غیرعمدی یک برنامه‌نویس یا توسط یک برنامه‌نویس غیرهمکار به وجود بیاید.

- فرض کنید که یک برنامه، ترتیب عملیات wait() و signal() را که بر روی سمافور mutex اجرا می‌شوند، جابه‌جا می‌کند که حاصل کار به صورت زیر خواهد شد:

```
signal(mutex);
          . . .
       critical section
          . . .
wait(mutex);
```

در این شرایط، چندین پردازه ممکن است با نقض نیاز انحصار متقابل، همزمان در حال اجرای بخش بحرانی خود باشند. این خطا امکان دارد تنها زمانی آشکار شود که چندین پردازه همزمان در بخش بحرانی خودشان اجرا دارند. توجه کنید که ممکن است این وضعیت همواره برگشت‌پذیر نباشد.

- فرض کنید یک پردازه، signal(mutex) را با wait(mutex) جایگزین می‌کند. یعنی، دنباله‌ی زیر را اجرا می‌کند:

```
wait(mutex);
          . . .
        critical section
          . . .
wait(mutex);
```

در این حالت، پردازه به‌صورت دایمی در فراخوانی دوم به ()wait مسدود خواهد ماند، چرا که اکنون سمافور در دسترس نیست.

- فرض کنید پردازه‌ای، wait(mutex) یا signal(mutex)، یا هر دو را حذف می‌کند. در این حالت، انحصار متقابل نقض می‌شود و یا پردازه به‌صورت دایمی مسدود مسدود خواهد ماند.

این مثال‌ها روشن می‌سازد که انواع مختلفی از خطاها می‌تواند به‌سادگی با استفاده‌ی نادرست سمافورها یا قفل‌های انحصار متقابل توسط برنامه‌نویسان در حل مسأله‌ی بخش بحرانی پیش بیاید. یک راهبرد برای مواجهه با این قبیل خطاها کارگذاشتن ابزار همگانی ساده به‌صورت ساختارهای سطح بالای زبان است. در این بخش، یک ساختار همگامی سطح بالای مبنایی را مطرح می‌کنیم—نوع **مانتیور**.

۶-۷-۱ استفاده از مانیتور

نوع داده‌ی انتزاعی—یا ADT—داده‌ها را که مستقل از هرگونه پیاده‌سازی خاص هستند با یک مجموعه توابع که بر روی آنها عملیات دارند، پنهان می‌کند. نوع مانیتور، یک ADT است که مجموعه عملیات تعریفی کاربر را که با انحصار متقابل در داخل مانیتور فراهم می‌شوند در بر دارد. نوع مانیتور، متغیرهایی را اعلان می‌کند که مقادیر آنها، حالت نمونه‌ای از آن نوع را به همراه بدنه‌ی توابعی که بر روی آن متغیرها عملیات دارند تعریف می‌کنند. ترکیب یک نوع مانیتور در شکل ۶-۱۱ نشان داده می‌شود. پردازه‌های مختلف نمی‌توانند مستقیماً از تشکیلات یک نوع مانیتور استفاده کنند. بنابراین، تابعی که در داخل یک مانیتور تعریف می‌شود تنها می‌تواند به متغیرهای محلی داخل مانیتور و پارامترهای صوری آن دسترسی داشته باشد. به طور مشابه، متغیرهای محلی یک مانیتور فقط قابل دسترس توابع محلی هستند.

ساختار مانیتور تضمین می‌کند که تنها یک پردازه می‌تواند در هر لحظه در داخل مانیتور فعال باشد. در نتیجه، برنامه‌نویس لازم ندارد که آشکارا برای این محدودیت همگامی نیز کد بنویسد (شکل ۶-۱۲). با این وجود، بنای مانیتور با تعریف اخیری که دارد، برای مدل‌سازی برخی طرح‌های همگامی از قدرت کافی برخوردار نیست. در راستای همین هدف، نیاز به تعریف راهکارهای همگامی اضافه‌تری داریم. این راهکارها توسط بنای شرط[1] فراهم می‌شوند. هر برنامه‌نویس که نیاز به نوشتن یک طرح همگامی مناسب دارد می‌تواند یک یا چند متغیر نوع *شرط* تعریف کند:

```
monitor monitor_name
{
```

1. Condition construct

```
        /* shared variable declarations */
        function P1 ( . . . ) {
                . . .
        }

        function P2 ( . . . ) {
                . . .
        }
                .
                .
                .
        function Pn ( . . . ) {
                . . .
        }

        initialization_code ( . . . ) {
                . . .
        }
}
```

شکل ۶-۱۱ نحو شبه‌کد یک مانیتور.

```
condition x, y;
```

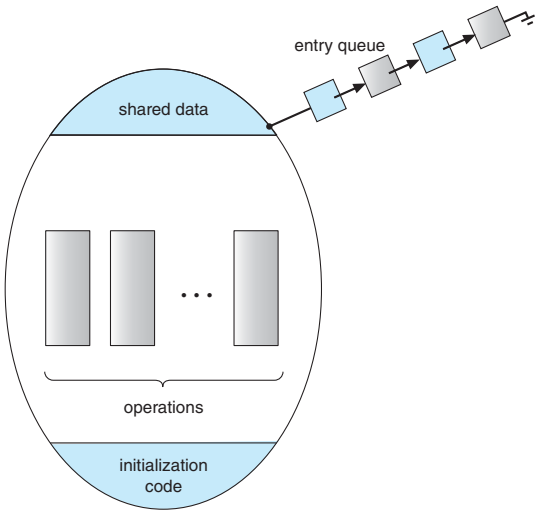
تنها عملیاتی که می‌توانند بر روی یک متغیر شرط احضار شوند عبارتند از: wait() و signal(). عملیات

```
x.wait();
```

به این معنی است که، پردازه‌ای که این عملیات را احضار کرده، معلق خواهد ماند تا این که، پردازه‌ای دیگر، عملیات

```
x.signal();
```

را احضار نماید.



شکل ۶-۱۲ چشم‌انداز شماتیک یک مانیتور.

عملیات x.signal() دقیقاً یکی از پردازه‌های معلق را از سر می‌گیرد. اگر هیچ پردازه‌ای معلق نباشد، در این صورت عملیات signal() بی‌اثر خواهد بود؛ یعنی، حالت x چنان خواهد ماند که گویی هرگز عملیاتی روی آن اجرا نشده است (شکل ۶-۱۳). این عملیات را با عملیات signal() مربوط به سمافورها مقایسه کنید که همواره روی حالت سمافور تأثیر می‌گذارد.

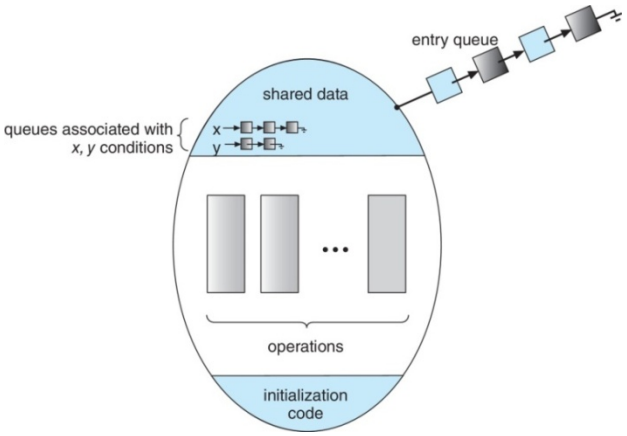
اکنون فرض کنید، لحظه‌ای که عملیات x.signal() توسط پردازه‌ی P احضار می‌شود، پردازه‌ای به نام Q وجود دارد که مرتبط با شرط x معلق است. روشن است که، اگر پردازه‌ی Q که معلق بوده، اجرای خود را از سر بگیرد، بایستی پردازه‌ی سیگنال‌دهنده، یعنی P منتظر بماند. در غیر این صورت، P و Q هر دو همزمان در داخل مانیتور فعال خواهند بود. اما، توجه داشته باشید که هر دو پردازه می‌توانند از نظر مفهومی، اجرای خود را ادامه بدهند. دو حالت امکان‌پذیر است:

۱- **سیگنال و انتظار**: P منتظر می‌ماند تا این که Q مانیتور را ترک کند و یا به انتظار شرط دیگری می‌ماند.

۲- **سیگنال و تداوم اجرا**: Q منتظر می‌ماند تا این که P مانیتور را ترک کند و یا به انتظار شرط دیگری می‌ماند.

استدلال‌های قابل قبولی برای انتخاب هر یک از این گزینه‌ها وجود دارند. از یک سو، از آنجایی که P از قبل در مانیتور اجرا داشت، روش *سیگنال و تداوم اجرا* عاقلانه‌تر است. از سویی دیگر، اگر به نخ P اجازه بدهیم که تداوم یابد، در این صورت ممکن است در زمانی که Q اجرا را از سر می‌گیرد، دیگر شرط منطقی که Q به خاطر آن منتظر بود برقرار نشود. یک مصالحه نیز بین این دو گزینه وجود دارد: هر زمان که نخ P عملیات سیگنال را اجرا می‌کند، بلافاصله مانیتور را ترک می‌کند. از این رو، Q بلافاصله از سر گرفته می‌شود.

بسیاری از زبان‌های برنامه‌نویسی، اعم از جاوا و C#، ایده‌ی مانیتور را آن‌گونه که در این بخش توصیف شده است با خود دارند. زبان‌های دیگر—همچون Erlang—پشتیبانی همروندی را با راهکاری مشابه فراهم می‌کنند.



شکل ۶-۱۳ مانیتور با متغیرهای شرط.

۶-۷-۲ پیاده‌سازی یک مانیتور با استفاده از سمافورها

اکنون یک پیاده‌سازی ممکن راهکار مانیتور را با استفاده از سمافورها مطرح می‌کنیم. برای هر مانیتور، سمافور دودویی mutex (با مقدار اولیه‌ی ۱) برای تضمین انحصار متقابل فراهم می‌شود. هر پردازه بایستی wait(mutex) را قبل از ورود به مانیتور و signal(mutex) را پس از ترک مانیتور اجرا نماید.

ما از طرح سیگنال-و-انتظار در پیاده‌سازی خودمان استفاده خواهیم کرد. از آنجایی که پردازه‌ی سیگنال‌دهنده بایستی منتظر بماند تا پردازه‌ی از سرگرفته‌شده، مانیتور را ترک و یا منتظر شود، سمافور دودویی دیگری به‌نام next، با مقدار اولیه‌ی ۰ معرفی می‌شود. پردازه‌های سیگنال‌دهنده می‌توانند از next برای تعلیق خودشان استفاده کنند. یک متغیر صحیح به نام next_count هم فراهم می‌شود تا تعداد پردازه‌های معلق را بر روی next بشمارد. بنابراین، هر تابع خارجی F با کد زیر جایگزین می‌شود:

```
wait(mutex);
        ...
  body of F
        ...
if (next_count > 0)
        signal(next);
else
        signal(mutex);
```

انحصار متقابل درون مانیتور تضمین است.

اکنون می‌توانیم چگونگی پیاده‌سازی متغیرهای شرط را نیز تشریح کنیم. برای هر متغیر شرط x سمافور دودویی x_sem و متغیر صحیح x_count را که هر دو مقدار اولیه‌ی ۰ می‌گیرند، معرفی می‌کنیم. اکنون، عملیات x.wait() می‌تواند به صورت زیر پیاده‌سازی شود:

```
x_count++;
if (next_count > 0)
        signal(next);
else
        signal(mutex);
wait(x_sem);
x_count--;
```

عملیات x.signal() می‌تواند به صورت زیر پیاده‌سازی شود:

```
if (x_count > 0) {
        next_count++;
        signal(x_sem);
        wait(next);
        next_count--;
}
```

این پیاده‌سازی برای تعاریف مانیتور ارائه شده توسط هر دوی Hoare و Brinch-Hansen قابل اعمال است. با این وجود، در برخی موارد، عمومیت پیاده‌سازی ضرورتی نداشته و یک بهبود قابل توجه در کارآمدی ممکن است. این مسأله را در تمرین ۶-۲۷ برای شما واگذار می‌کنیم.

۶-۷-۳ از سرگیری پردازه‌ها در داخل مانیتور

اکنون به موضوع ترتیب از سرگیری پردازه‌ها در داخل مانیتور بر می‌گردیم. اگر چندین پردازه بر روی شرط x معلق باشند و یک عملیات x.signal() توسط پردازه‌ای اجرا شود، در این صورت چگونه تعیین کنیم که در ادامه، کدام یک از پردازه‌های معلق از سرگرفته شود؟ یک راه‌حل ساده آن است که از یک ترتیب اولین-ورود، اولین-سرویس (FCFS) استفاده کنیم، لذا نخست پردازه‌ای که طولانی‌ترین انتظار را داشته است، از سرگرفته می‌شود. با این وجود، در بسیاری از شرایط، استفاده از چنین طرح ساده‌ای کافی نیست. در این شرایط، ساختار **انتظار شرطی**[1] می‌تواند استفاده شود. این ساختار شکل زیر را دارد:

```
x.wait(c);
```

که در آن c عبارت صحیحی است که در زمان اجرای عملیات wait() ارزیابی می‌شود. سپس مقدار c، که **شماره‌ی الویت** نامیده می‌شود به همراه نام پردازه‌ای که معلق می‌شود، ذخیره می‌گردد. زمانی که x.signal() اجرا می‌شود، پردازه‌ای که شماره‌ی الویت مربوط به آن کوچکترین مقدار را دارد از سرگرفته می‌شود.

1. Conditional wait construct

برای روشن‌شدن این راهکار جدید، مانیتور ResourceAllocator را که در شکل ۶-۱۴ نشان داده شده و تخصیص یک منبع منفرد را مابین پردازه‌های رقابت‌کننده کنترل می‌کند در نظر می‌گیریم. هر زمان که یک پردازه درخواستی برای کسب منبع صادر می‌کند، حداکثر زمانی را که برنامه‌ریزی کرده تا از منبع استفاده نماید، مشخص می‌کند. مانیتور، منبع را در اختیار پردازه‌ای می‌گذارد که کوتاهترین زمان تخصیص را درخواست داده باشد. هر پردازه که به دسترسی منبع مطلوب نیاز دارد بایستی دنباله‌ی زیر را پشت سر بگذارد:

```
R.acquire(t);
        ...
  access the resource;
        ...
R.release();
```

که R، یک نمونه از نوع ResourceAllocator است.

```
monitor ResourceAllocator
{
        boolean busy;
        condition x;

        void acquire(int time) {
                if (busy)
                        x.wait(time);
                busy = true;
        }

        void release() {
                busy = false;
                x.signal();
        }

        initialization_code() {
                busy = false;
        }
}
```

شکل ۶-۱۴ مانیتوری برای تخصیص یک منبع منفرد.

متأسفانه، مفهوم مانیتور نمی‌تواند تضمین کند که دنباله‌ی دسترسی مذکور رعایت خواهد شد. به‌خصوص، مسائل زیر هم ممکن است پیش بیایند:

- امکان دارد پردازه‌ای اقدام به دسترسی منبعی نماید، بی آنکه نخست مجوز دسترسی منبع را به دست آورد.
- امکان دارد پردازه‌ای با در اختیارگرفتن دسترسی یک منبع، هرگز آن را رها نکند.
- امکان دارد پردازه‌ای اقدام به رهایی منبعی نماید که هرگز آن را درخواست نداده است.

- امکان دارد پردازه‌ای منبع یکسانی را برای بار دوم (بدون این که، اول آن منبع را رها کند) درخواست نماید.

مشکلاتی مشابه این مسائل، در رابطه با استفاده از سمافورها نیز وجود داشتند و این مشکلات در ذات خود مشابه آنهایی است که در ابتدا ما را تشویق کردند تا بناهای مانیتور را توسعه بدهیم. پیشتر دغدغه‌ی استفاده‌ی درست سمافورها را داشتیم. اکنون، بایستی نگران استفاده‌ی درست عملیات سطح بالای تعریفی برنامه‌نویس باشیم که دیگر کامپایلر هم نمی‌تواند کمکی بکند.

یک راه‌حل ممکن مسأله‌ی فعلی این است که عملیات دسترسی منبع را در داخل مانیتور ResourceAllocator قرار بدهیم. با این وجود، استفاده از این راه‌حل بدان معنی است که زمانبندی به جای آنکه از آنچه کد کرده‌ایم صورت بگیرد، مطابق الگوریتم زمانبندی تعبیه‌شده در داخل مانیتور انجام خواهد گرفت.

برای این که مطمئن شویم، پردازه‌ها دنباله‌های مناسبی را طی می‌کنند، بایستی تمامی برنامه‌هایی را که از مانیتور ResourceAllocator و منبع تحت مدیریت آن استفاده می‌کنند، بازرسی کنیم. برای نهادینه‌کردن صحت این سیستم لازم است دو شرط را بررسی کنیم. اولاً، پردازه‌های کاربر بایستی همواره فراخوانی‌های مانیتور خود را با دنباله‌ی درستی انجام بدهند. ثانیاً، باید مطمئن شویم که یک پردازه‌ی غیرهمکار به راحتی نمی‌تواند با نادیده گرفتن دروازه‌ی انحصار متقابل مانیتور و بدون استفاده از پروتکل‌های دسترسی، یک منبع مشترک را مستقیماً دستیابی کند. تنها با برقراری این دو شرط است که می‌توان مطمئن بود هیچ‌گونه خطای زمانی رخ نخواهد داد و الگوریتم زمانبندی با شکست مواجه نخواهد شد.

اگرچه این بازرسی در یک سیستم کوچک و ایستا امکان‌پذیر است، برای یک سیستم بزرگ یا یک سیستم پویا نمی‌تواند روش مقبولی باشد. مسأله‌ی کنترل دسترسی فقط می‌تواند با استفاده از راهکارهای اضافه‌تری حل شود که در فصل ۱۷ توصیف می‌شوند.

۶-۸ عاقبت به‌خیری

یک پیامد استفاده از ابزار همگامی برای هماهنگی دسترسی به بخش بحرانی، این احتمال است که پردازه‌ای که تلاش دارد به بخش بحرانیش وارد شود به‌صورت نامتناهی منتظر بماند. از بخش ۶-۲ به‌خاطر بیاورید که ما سه معیار مطرح کردیم که راه‌حل‌های مسئله‌ی بخش بحرانی بایستی ارضا نمایند. انتظار نامتناهی دو مورد از این‌ها—معیارهای پیشروی و انتظار محدود—را نقض می‌کند.

عاقبت به‌خیری[1] به مجموعه‌ای از ویژگی‌ها اطلاق می‌شود که یک سیستم بایستی ارضا نماید تا تضمین شود که پردازه‌ها در طی چرخه‌ی حیات اجرایشان پیشروی دارند. انتظار نامتناهی یک پردازه تحت شرایطی که بیان شد مثالی از یک «شکست عاقبت به‌خیری[2]» است.

1. Liveness
2. Liveness failure